

التحسينات التقنية المفصلة - منصة كلمات

تحسينات أمان الكود 1.

SQL Injection (أ) حماية من هجمات

```
// ❌ خطر - لا تفعل هذا
const query = `SELECT * FROM
users WHERE id = ${userId}`;

// ✅ parameterized آمن - استخدم
queries
const query =
db.collection('users').doc(userId
).get();
```

XSS (Cross-Site Scripting) (ب) حماية من

```
// ❌ خطر
element.innerHTML = userInput;

// ✅ استخدام textContent - آمن
sanitize
element.textContent = userInput;
// أو استخدام مكتبة مثل DOMPurify
```

ج) حماية من CSRF:

```
// إلى كل طلب CSRF token أضف
fetch('/api/delete-post', {
  method: 'POST',
  headers: {
    'X-CSRF-Token': csrfToken,
    'Content-Type':
'application/json'
  }
});
```

د) تشفير البيانات الحساسة

```
// bcrypt كلمات المرور - استخدم
const hashedPassword = await
```

```
bcrypt.hash(password, 10);

// encryption الرسائل الخاصة - استخدم
const encrypted = await
encryptMessage(message,
recipientPublicKey);
```

تحسينات الأداء 2.

أ) Lazy Loading للصور:

```
<!-- استخدم native lazy loading -->


<!-- استخدم Intersection
Observer API -->

```

```
const imageObserver = new
IntersectionObserver((entries) =>
```

```

{
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      entry.target.src =
entry.target.dataset.src;

      imageObserver.unobserve(entry.target);
    }
  });
});

document.querySelectorAll('img.lazy').forEach(img => {
  imageObserver.observe(img);
});

```

ب) ضغط الصور عند الرفع:

```

// أو Sharp استخدم مكتبة مثل
ImageMagick
const sharp = require('sharp');

const compressImage = async
(buffer) => {

```

```
return sharp(buffer)
    .resize(1920, 1080, { fit:
'inside', withoutEnlargement:
true })
    .webp({ quality: 80 })
    .toBuffer();
};
```

الذكي Caching ج)

```
// استخدم HTTP caching headers
app.get('/api/posts', (req, res)
=> {
    res.set('Cache-Control',
'public, max-age=300'); // 5 دقائق
    res.json(posts);
});

// لا Service Worker أو استخدم cache
if ('caches' in window) {
    caches.open('v1').then(cache =>
{
    cache.add('/posts');
    });
}
```

د) Minification و Tree Shaking:

```
// في webpack.config.js
module.exports = {
  mode: 'production',
  optimization: {
    usedExports: true, // Tree
shaking
    minimize: true,
  },
  output: {
    filename: 'bundle.min.js'
  }
};
```

هـ) للملفات الثابتة CDN (هـ):

```
<!-- استخدم CDN لـ CSS و JavaScript
-->
<script
src="https://cdn.example.com/boot
strap.min.js"></script>
```

تحسينات قاعدة البيانات (Firebase)

أ) Indexes المحسنة:

```
// للحقول المستخدمة في indexes أضف الاستعلامات
db.collection('posts')
  .where('userId', '==', userId)
  .where('timestamp', '>=',
oneWeekAgo)
  .orderBy('timestamp', 'desc')
  .limit(10)
  .get();

// Firebase سيطلب منك إنشاء index
```

ب) Pagination:

```
// لتقليل البيانات pagination استخدم
المحملة
let lastDoc = null;

const loadMorePosts = async () =>
```

```
{
  let query =
    db.collection('posts').orderBy('timestamp', 'desc').limit(20);

  if (lastDoc) {
    query =
      query.startAfter(lastDoc);
  }

  const snapshot = await
    query.get();
  lastDoc =
    snapshot.docs[snapshot.docs.length - 1];

  return snapshot.docs.map(doc =>
    doc.data());
};
```

ج) Batch Operations:

```
// لعدة عمليات معاً batch استخدم
const batch = db.batch();
```

```
batch.set(db.collection('users').
doc(userId), userData);
batch.update(db.collection('posts
').doc(postId), { commentCount:
increment(1) });
batch.delete(db.collection('notif
ications').doc(notifId));

await batch.commit();
```

4. تحسينات التصميم المتجاوب (Responsive Design)

أ) CSS Grid و Flexbox:

```
/* Grid تخطيط متجاوب باستخدام */
.posts-container {
  display: grid;
  grid-template-columns:
repeat(auto-fit, minmax(300px,
1fr));
  gap: 20px;
}
```

```
/* شريط جانبي على الشاشات الكبيرة */  
@media (min-width: 1024px) {  
  .main-layout {  
    display: grid;  
    grid-template-columns: 1fr  
300px;  
  }  
}
```

ب) Mobile-First Approach:

```
/* mobile ابدأ بـ */  
.post {  
  font-size: 14px;  
  padding: 10px;  
}  
  
/* أضف تحسينات للشاشات الأكبر */  
@media (min-width: 768px) {  
  .post {  
    font-size: 16px;  
    padding: 20px;  
  }  
}
```

```
@media (min-width: 1024px) {  
  .post {  
    font-size: 18px;  
    padding: 30px;  
  }  
}
```

ج) Viewport Meta Tag:

```
<meta name="viewport"  
content="width=device-width,  
initial-scale=1.0">
```

5. UX/UI تحسينات

أ) Loading States:

```
<!-- أظهار حالة التحميل للمستخدم -->  
<div class="loading-skeleton">  
  <div class="skeleton-text">  
</div>  
  <div class="skeleton-avatar">
```

```
</div>  
</div>
```

ب) Error Handling:

```
try {  
  const response = await  
  fetch('/api/posts');  
  if (!response.ok) {  
    throw new Error(`HTTP error!  
status: ${response.status}`);  
  }  
  const data = await  
  response.json();  
  displayPosts(data);  
} catch (error) {  
  showErrorMessage('حدث خطأ. حاول  
مجدداً لاحقاً');  
  console.error('Error:', error);  
}
```

ج) Form Validation:

```
// التحقق من الصيغ قبل الإرسال  
function validateEmail(email) {
```

```

    const regex =
    /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    return regex.test(email);
}

function
validatePassword(password) {
    return password.length >= 8 &&
        /[A-Z]/.test(password)
    &&
        /[a-z]/.test(password)
    &&
        /[0-9]/.test(password);
}

```

د) Debouncing للبحث:

```

// تجنب الاستعلامات المتكررة
const debounce = (func, wait) =>
{
    let timeout;
    return function(...args) {
        clearTimeout(timeout);
        timeout = setTimeout(() =>
func(...args), wait);
    }
}

```

```
};  
};  
  
const handleSearch =  
debounce((query) => {  
  searchUsers(query);  
}, 500); // 500ms قبل البحث
```

6. PWA (Progressive Web App)

i) Service Worker:

```
// sw.js  
const CACHE_NAME = 'kalimat-v1';  
const urlsToCache = [  
  '/',  
  '/index.html',  
  '/styles.css',  
  '/app.js',  
  '/images/icon-192x192.png'  
];
```

```
self.addEventListener('install',
event => {
    event.waitUntil(

    caches.open(CACHE_NAME).then(cache
e => {
        return
        cache.addAll(urlsToCache);
    })
    );
});

self.addEventListener('fetch',
event => {
    event.respondWith(

    caches.match(event.request).then(
response => {
        return response ||
        fetch(event.request);
    })
    );
});
```

ب) Manifest.json:

```
{
  "name": "منصة كلمات",
  "short_name": "كلمات",
  "description": "منصة عربية اجتماعية",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#0066cc",
  "icons": [
    {
      "src": "/images/icon-192x192.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "/images/icon-512x512.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

Service Worker: (ج) تسجيل

```
// في app.js
if ('serviceWorker' in navigator)
{

navigator.serviceWorker.register(
  '/sw.js').then(() => {
    console.log('Service Worker
registered');
  });
}
```

7. المراقبة والتحليلات الآمنة

(أ) تتبع الأخطاء

```
// أو مشابه Sentry استخدم
Sentry.init({
  dsn: 'https://your-sentry-url',
  environment: 'production'
});
```

```
window.addEventListener('error',  
(event) => {  
  
  Sentry.captureException(event.error);  
});
```

ب) قياس الأداء

```
// استخدم Performance API  
const perfData = {  
  pageLoadTime:  
    performance.timing.loadEventEnd -  
    performance.timing.navigationStart,  
  resourceLoadTime:  
    performance.timing.responseEnd -  
    performance.timing.fetchStart,  
  renderTime:  
    performance.timing.domContentLoadedEventEnd -  
    performance.timing.navigationStart  
};
```

```
console.log('Performance:',  
perfData);
```

ج) التحليلات الخاصة

```
// تتبع الأحداث بدون تجسس  
analytics.track('post_created', {  
  postType: 'text', // لا ترسل محتوى  
  الملاحظات  
  timestamp: new Date(),  
  isPublic: true  
});
```

8. التوثيق والأفضليات

أ) للتوثيق JSDoc:

```
/**  
 * ينشئ منشوراً جديداً  
 * @param {string} content - محتوى  
 المنشور  
 * @param {string[]} images -  
 مصفوفة من روابط الصور
```

```
* @param {boolean} isPublic - هل
المنشور عام؟
* @returns {Promise<Object>}
المنشور المنشأ
*/
async function
createPost(content, images = [],
isPublic = true) {
  // implementation
}
```

ب) Git Commit Messages:

✨ feat: أضف ميزة حذف الحساب

- إضافة صفحة إعدادات جديدة
- تنبيهات واضحة عن الحذف
- تأكيد من خطوتين

Fixes #123

ج) README.md شامل:

منصة كلمات

.منصة عربية اجتماعية للنقاش الفكري

التثبيت

1. استنسخ المستودع
2. `npm install` : ثبت المتطلبات
3. `npm run setup-env` : اجعل ملف .env

الاستخدام

...

الخلاصة:

هذه التحسينات تضمن منصة آمنة وسريعة وسهلة
استخدام 🙌